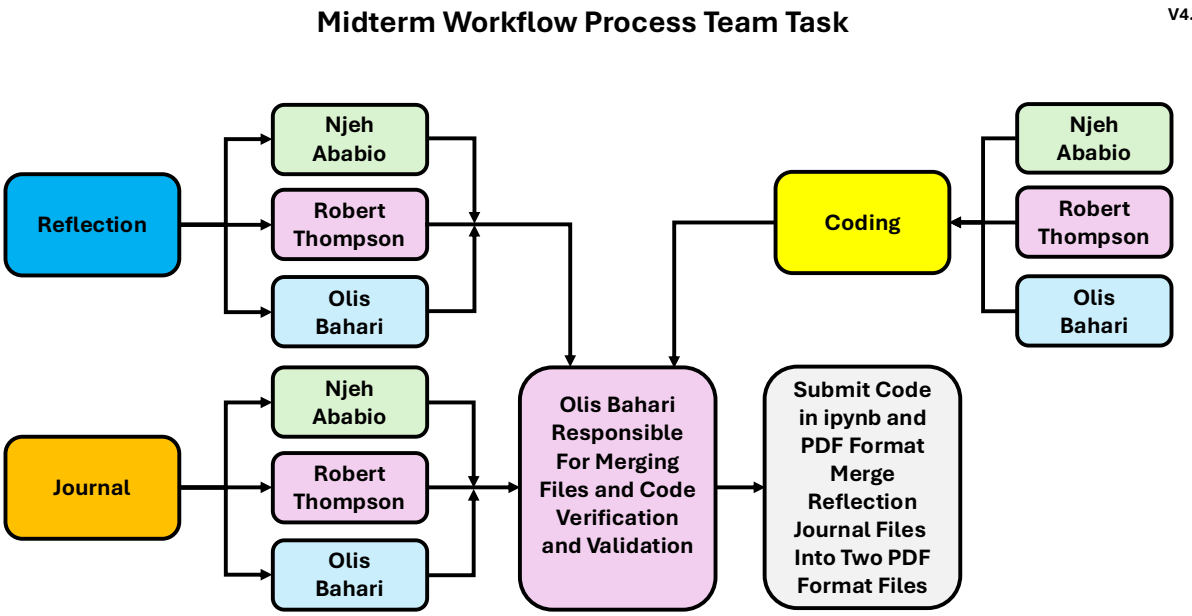
Midterm Journal Merged Njeh, Robert, & Olis ITAI2376



Njeh Ababio: Diffusion Model Learning Journal

Learning focus

This journal outlines what I learned from the diffusion model assignment and how the project helped me improve in coding and problem-solving. The most important thing I learned is that image generation involves a systematic process: adding noise to an image during training and having the model learn to predict that noise, enabling gradual reconstruction.

Understanding diffusion and the U-Net

I found out that in forward diffusion, noise is slowly added to an image. During training, the model is given an image and predicts the noise; the loss function then compares the predicted noise with the real noise, which helped me better understand what the model is learning and how the training objective relates to image generation.

I also looked into the reverse process, this process starting with random noise before the step-by-step formation of an image. After having examined the structure of the U-Net, I was able to grasp its design, namely that the down sampling layers pick up more general features, the up-sampling layers restore the spatial details, and the skip connections transmit information between the corresponding encoding and decoding layers so as to maintain the useful image features during reconstruction.

Conditioning and evaluating image quality

The model knows which diffusion step it is currently processing through the use of time conditioning and receives a regression-type signal via class conditioning. Now that I can distinguish between the two kinds of input, I can see how the model gets guidance when making the noise prediction.

I concluded that a low training loss does not ensure that the generated images are recognizable; although better noise prediction is useful evidence, the generated images have to be examined and reviewed to assess their quality and to check that they match the intended class. I have also found that CLIP offers another way to evaluate how well an image matches a text description; I will consider the score alongside the images and other evidence rather than treating it as a full assessment.

Coding and problem solving

Since I have become more conscious of all the different details that affect a deep learning implementation—for example, data normalization, tensor dimensions, device placement, and GPU memory—it has become clear to me that these factors can influence not only the correct operation of the code but also the model's performance, and as a result I now understand how important it is to test each component and to verify the intermediate results before carrying out full training.

Understanding how my section fits into the whole notebook also helped me clarify my explanation of its purpose. My objective is to link each section of the code to the data it receives, the result it produces, and its role in the overall diffusion process; this method improves the process, makes it more systematic, and strengthens my ability to discuss the implementation.

Plans for future improvement

For my future work, I intend to draw up some pseudocode in order to explain the various sections of the notebook, and I will keep a record of the experimental conditions so that I can check how each experiment was set up; by comparing the samples taken at different stages of training, I will be able to evaluate the changes in image quality, and by saving the checkpoints, I can monitor my progress and keep useful model states.

What I would like to do now is to combine a deeper conceptual understanding with a more structured coding workflow; by keeping a record of my experiments, checking the different components, and looking at both the loss values and the generated samples, I hope to become better at spotting problems and to work with deep learning models with greater confidence.

Robert Thompson: Diffusion Model Coding Journal

Focus of my coding reflection

In this journal, I am going to talk about the techniques that I applied and the different problems I came across during my work on the diffusion model project; my main objective has been to ensure that the implementation is easy for anyone to understand, check, and reproduce. Having looked at the project in its current form, I found several areas in which it could be improved, especially regarding its organization, the tensor checks, reliable training, and experiment logging; these improvements should also prove useful to the reviewers when they are examining the code or when they want to extend it.

Organizing the implementation

It would be helpful if the different stages—such as dataset preparation, the U-Net components, the noise schedule, training, and sampling—had been clearly defined as separate functions, since this would make it easier to see how a change in one of them affects the others. For this reason, my first action will be to specify the inputs and outputs for each component.

I also intend to provide adjustable settings in the configuration file and, in the code, will include an explanation of the key tensor shapes together with the reasons for the major operations; thus, I will be able to implement the code with greater confidence, and it will be easier for my teammates to understand the changes.

Checking tensors and diffusion calculations

Prior to carrying out the full training process, I must examine the image dimensions, the device setup, the class labels, and the time-step ranges; I also have to make sure that the magnitude of the predicted noise matches that of the target noise and that the loss is finite, and it is only after these checks have been carried out that I should do the preliminary runs in order to verify the conditions before using the up sources in the longer experiment.

It is just as crucial to check the diffusion calculations; I have to ensure that the forward noise formula and the reverse sampling updates actually make use of the correct noise schedule coefficients, because if they don't the code might appear to work but in fact it would not be carrying out the right calculation. I should be able to explain why each coefficient is used and also have to verify that the implementation corresponds to the intended process.

Training and sampling practices

In order to do this, I maintain a record of the model, the optimizer, and the configuration details at every training update. When it comes to evaluation, I cease tracking the unnecessary components. Regarding the label, I have to think about it carefully rather than simply assume that it is always beneficial. I had to look into its effect on speed, memory usage, and numerical stability. I aim to choose settings that are suitable for my resources without compromising implementation reliability, record-keeping, or debuggability.

In all the experiments, I recorded the library versions in which certain operations remained nondeterministic. Even if I cannot guarantee that the experiments can be reproduced, the results do provide a basis for reproducing them.

In order that the various kinds of problems which could occur during development may be made clear, I have included some log messages together with example previews. As I continue with the debugging process, I will use these records to connect the configurations with the behavior that is observed. Since it is hard to determine which changes made a difference, I will carry out controlled comparisons whenever this is possible and will clearly document each adjustment. Steps for improvement

I also plan on dividing the code into separate sections, carrying out an initial check, and making sure that the records remain consistent. In the meantime, I hope to improve my explanation of the forward and reverse diffusion processes so that, as I look at each section, I will be able to give you a clearer one for your review and further development.

Olis Bahari: Diffusion Model Project Journal

Project focus and personal learning

This journal outlines what I have learned and how I intend to improve the diffusion model midterm project. My primary conclusion is that I need to combine a reliable implementation with controlled training experiments and comparisons. To explain why a model produces certain images, I must connect its internal mechanisms to evidence in its outputs. Only by taking this approach can I make better decisions about image quality and computational resource use.

Checking the data and model inputs

An essential part of my method involves reviewing the data and model inputs before investing resources in the next stage; in this case, I pass a small portion of the data through the model to verify image normalization, class labels, and time-step inputs. I also make sure that the predicted noise has the same shape as the target noise. This will let me catch implementation errors early and give me a clearer starting point when troubleshooting.

Because of this planning, I have concluded that preparation is essential to the experiment; if the inputs are wrong, the problem may not be solvable by altering the training settings. I intend to carry out these checks as part of my normal working procedure so that, when I finally look at the results, I can do so with greater confidence.

Recording and comparing experiments

I keep a record of the settings, training loss, sample images, and checkpoints. This lets me review an experiment and clearly understand the conditions that led to its outputs. The checkpoints also permit me to restart the training process. In the future, I will change only one setting at a time—for example, the learning rate, the batch size, or the noise schedule—while keeping all other settings the same. I will record each change, along with the loss and the generated examples. As a result, it will be easier to determine which changes improve performance and which settings require further investigation.

Evaluating generated images

When I conduct my evaluation, I select a fixed number of examples for each category and make sure the images are easy to identify, show a good variety of differences, and are indeed the correct class. Because I choose examples from a wide range of categories, my assessment of the model isn't based on a small number of especially attractive outputs.

I look at the different stages of denoising and evaluate the CLIP similarity scores based on what I can see. Because a single score cannot capture all aspects of image quality, I also look for visible defects and check that the results are consistent across samples. In future reports, we will apply the same evaluation method to all our experiments so the comparisons are more meaningful.

Resource management and next steps

For my next experiment, I will begin with a small dataset and a simulation, and only proceed to more complicated setups when my resources allow; if I run out of GPU memory, I will first reduce the batch size and then use a smaller model if necessary. This approach lets me do practical testing with the hardware I have.

I also intend to look into time embeddings and the reverse-sampling equations in more detail, because only then can I explain the denoising process and debug it more efficiently. My aim is to set up a procedure that lets me check the inputs, run controlled experiments, evaluate the outputs consistently, and explain my technical decisions.